

Wielkoskalowe uczenie maszynowe i metoda stochastycznego spadku wzdłuż gradientu

W poprzednim podrozdziale nauczyliśmy się minimalizować funkcję kosztu poprzez wykonywanie kroku oddalającego od gradientu obliczonego z całego zbioru danych uczących; dlatego algorytm ten jest czasami nazywany **wsadową** metodą gradientu prostego. Wyobraź sobie teraz, że masz do dyspozycji olbrzymi zestaw danych zawierający miliony punktów danych, co jest dość często spotykaną sytuacją w technikach uczenia maszynowego. W takim przypadku stosowanie metody wsadowej gradientu bywa dość kosztowne pod względem obliczeniowym, ponieważ musimy od nowa oceniać cały zbiór danych uczących za każdym razem, gdy wykonujemy kolejny krok w kierunku globalnego minimum.

Popularnym zamiennikiem algorytmu wsadowego gradientu prostego jest metoda **stochastycznego spadku wzdłuż gradientu** (ang. *stochastic gradient descent*), czasami nazywana także **iteracyjnym algorytmem spadku wzdłuż gradientu**. Nie aktualizujemy w tej sytuacji wag na podstawie sumy nagromadzonych błędów spośród wszystkich próbek $\mathbf{x}^{(i)}$:

$$\Delta \mathbf{w} = \eta \sum_i (y^{(i)} - \phi(z^{(i)})) \mathbf{x}^{(i)}$$

Aktualizujemy wagi przyrostowo dla każdej próbki uczącej:

$$\eta (y^{(i)} - \phi(z^{(i)})) \mathbf{x}^{(i)}$$

Chociaż algorytm stochastycznego spadku wzdłuż gradientu można uznawać za aproksymację gradientu prostego, zazwyczaj umożliwia on znacznie szybsze uzyskanie zbieżności, gdyż wagi są częściej aktualizowane. Każdy gradient jest wyliczany na podstawie pojedynczej próbki uczącej, dlatego powierzchnia błędów generuje większe szумы niż w gradiencie prostym, co również ma znaczenie, gdyż dzięki temu algorytm stochastyczny łatwiej może ignorować płytkie minima lokalne. Aby otrzymać dokładne wyniki za pomocą stochastycznego spadku wzdłuż

gradientu, bardzo ważne jest zaprezentowanie algorytmowi danych w przypadkowej kolejności, dlatego chcemy przed każdą epoką przetasować zbiór próbek, aby uniknąć cykliczności.

W implementacjach stochastycznego spadku wzdłuż gradientu niezmienny współczynnik uczenia η często jest zastępowany adaptacyjnym współczynnikiem uczenia, którego wartość maleje wraz z upływem czasu; może on przybrać np. następującą postać: $\frac{c_1}{[liczba\ iteracji] + c_2}$, gdzie c_1 i c_2 są stałymi. Zwróć uwagę, że algorytm ten nie osiąga globalnego minimum, lecz dociera do jego bardzo zbliżonych wartości. Dzięki adaptacyjnemu współczynnikowi uczenia możemy jeszcze bardziej zbliżyć się do globalnego minimum.

Kolejną zaletą metody stochastycznego spadku wzdłuż gradientu jest możliwość wykorzystania jej do **uczenia przyrostowego**. Rozwiązanie to polega na trenowaniu modelu za pomocą ciągle napływających nowych danych uczących. Jest to technika przydatna zwłaszcza w sytuacji gromadzenia dużych ilości informacji — np. danych o użytkownikach w typowej aplikacji sieciowej. Poprzez uczenie w locie system jest w stanie natychmiastowo dostosować się do zmian, a dane uczące mogą zostać usunięte po zaktualizowaniu modelu, jeżeli pojemność dyskowa stanowi problem.

Kompromisem pomiędzy metodą gradientu prostego a stochastycznym spadkiem wzdłuż gradientu jest tzw. **uczenie z pomocą minipaczek** (ang. *mini-batch learning*). Rozwiązanie to można rozpatrywać jako stosowanie metody wsadowej gradientu do mniejszych podzbiorów danych uczących — np. 50 próbek. Zaletą uczenia z pomocą minipaczek jest znacznie szybsza konwergencja w porównaniu z gradientem prostym z powodu częstszych aktualizacji wag. Do tego metoda ta pozwala zastępować pętlę `for` używaną do próbek uczących w **stochastycznym spadku wzdłuż gradientu** operacjami wektorowymi, co jeszcze bardziej poprawia skuteczność obliczeniową danego algorytmu uczenia.

Wcześniej zaimplementowaliśmy regułę uczenia Adaline wykorzystującą metodę gradientu prostego, dlatego wystarczy wprowadzić do niej kilka modyfikacji, żeby algorytm zaczął aktualizować wagi poprzez stochastyczny spadek wzdłuż gradientu. Teraz wewnątrz metody `fit` będziemy aktualizować wagi po sprawdzeniu każdej próbki uczącej. Następnie zaimplementujemy dodatkową metodę `partial_fit`, która nie inicjuje od nowa wag w przypadku uczenia w locie. Aby sprawdzić zbieżność algorytmu po treningu, będziemy obliczać koszt jako średni koszt próbek uczących w każdej epoce. Ponadto dodamy możliwość tasowania (`shuffle`) danych uczących przed rozpoczęciem każdej epoki, dzięki czemu unikniemy cykliczności podczas optymalizowania funkcji kosztu; parametr `random_state` służy do generowania wartości losowej dla zachowania większej ciągłości: